

HELEP

L3 Patterns in Code Document

SEN3244 — Software Architecture | Engr. Tekoh Palma | May 2026

Introduction

This document shows where each design pattern lives in the HELEP codebase, what problem it solves, and includes annotated code snippets with file and line references. Six patterns were pre-implemented in the starter (A.1–A.6); two were added as part of this submission (B.1 CQRS and B.2 Bulkhead). One anti-pattern is documented in Part C to explain what was deliberately kept out.

Part A — Patterns in the Starter

A.1 — Choreographed Saga

Where: `sos-service/app/main.py` | `dispatch-service/app/main.py` | `notification-service/app/main.py`

Catalogue: [Enterprise Integration Patterns \(Hohpe & Woolf\)](#); [Microservices Patterns \(Richardson\)](#)

A Choreographed Saga coordinates a transaction that spans multiple services without putting a central orchestrator in charge. Each service listens for events, does its part, and publishes the next event. If something needs to be undone, a compensating event triggers the rollback.

In HELEP the full happy-path chain looks like this:

```
# 1. sos-service/app/main.py -- user fires SOS
insert_incident(iid, user_id, lat, lon, mode, media_ref) # save to DB
await publish("sos.triggered", {...}, key=iid)           # emit to Kafka

# 2. dispatch-service/app/main.py -- picks up sos.triggered
if stream == "sos.triggered":
    pick = matcher().pick(p["lat"], p["lon"], all_free_responders())
    if pick and reserve_responder_for(pick["id"], p["incident_id"]):
        await publish("responder.assigned", {...}, key=p["incident_id"])

# 3. notification-service/app/main.py -- picks up responder.assigned
if stream == "responder.assigned":
    log.info("sms.sent", to=victim_id, responder=r_id)
    insert_notification(...)
    await publish("notification.sent", {...}, key=iid)

# 4. analytics-service -- consumes every stream, increments counters
```

The cancel path works the same way in reverse. When a citizen cancels, `sos-service` emits `sos.cancelled`. `dispatch-service` picks it up, calls `release_assignment()` to free the responder, then publishes `responder.confirmed` with `status=RELEASED`.

```
# sos-service/app/main.py cancel_sos()
update_incident_status(iid, "CANCELLED")
await publish("sos.cancelled", {"incident_id": iid, "user_id": uid}, key=iid)

# dispatch-service/app/main.py on_event() -- handles sos.cancelled
elif stream == "sos.cancelled":
    release_assignment(p["incident_id"]) # db.py: sets busy=0 on responder
    await publish("responder.confirmed", {..., "status": "RELEASED"}, key=iid)
```

A.2 — Pub/Sub via Apache Kafka

Where: `app/events.py` in every service — `producer()` lines 14-27, `consume()` lines 80-98

Every service that needs to talk to another one does it by writing an event to a Kafka topic and reading from one. There are no direct HTTP calls between services at all. The broker holds messages until the consumer is ready, so a temporary outage in dispatch-service doesn't mean lost SOS events.

The consumer uses manual offset commit so the offset only moves forward after the handler runs successfully. If the handler crashes the message is re-delivered next time the pod comes up (at-least-once semantics).

```
# app/events.py consume()
consumer = AIOKafkaConsumer(
    *topics,
    bootstrap_servers=KAFKA_BOOTSTRAP,
    group_id=group_id,
    enable_auto_commit=False,          # we commit manually after success
    auto_offset_reset="earliest",      # replay from start if we've never read this partition
    value_deserializer=lambda raw: json.loads(raw.decode()),
)
await consumer.start()
async for msg in consumer:
    data = msg.value
    data["_stream"] = msg.topic
    try:
        await handler(data)
        await consumer.commit()        # only here does the offset move
    except Exception:
        pass # offset stays put -> same message arrives again next poll
```

A.3 — Repository

Where: `app/db.py` in every service (e.g. `dispatch-service/app/db.py`)

All database access is hidden behind named functions. The route handlers in `main.py` never write SQL directly — they call things like `reserve_responder_for()`, `insert_incident()`, `all_free_responders()`. This means we could switch from SQLite to PostgreSQL by changing only `db.py`, not any of the FastAPI files.

```
# dispatch-service/app/db.py reserve_responder_for()
def reserve_responder_for(rid: str, incident_id: str) -> bool:
    with conn() as c:
        # One atomic write: if another pod already claimed this responder,
        # rowcount will be 0 and we back off. This is the no-double-dispatch guarantee.
        cur = c.execute(
            "UPDATE responders SET busy = 1 WHERE id = ? AND busy = 0", (rid,)
        )
        if cur.rowcount == 0:
            return False
        try:
            c.execute(
                "INSERT INTO assignments (incident_id, responder_id, status)"
                " VALUES (?, ?, 'ASSIGNED')", (incident_id, rid)
            )
        except sqlite3.IntegrityError:
            c.execute("UPDATE responders SET busy = 0 WHERE id = ?", (rid,))
            return False
    return True
```

A.4 — Strategy

Where: `dispatch-service/app/matching.py` — all three classes + factory function

Strategy lets us swap the responder-selection algorithm at runtime without touching any other code. The Matcher protocol defines a single method pick() and three classes implement it. A factory function reads the MATCHER environment variable and returns the right one.

```
# dispatch-service/app/matching.py

class NearestMatcher:          # MATCHER=nearest (default)
    def pick(self, lat, lon, responders) -> dict | None:
        # walks the list and returns the one with smallest haversine distance

class CredibilityWeightedMatcher: # MATCHER=credibility
    def pick(self, lat, lon, responders) -> dict | None:
        # score = credibility / (distance_km + 1) -- higher score wins

class RoundRobinMatcher:      # MATCHER=roundrobin [added in this submission]
    _turn: int = 0
    def pick(self, lat, lon, responders) -> dict | None:
        # cycles through pool[_turn % len(pool)] -- fair workload distribution

def matcher() -> Matcher:
    choice = os.getenv("MATCHER", "nearest").lower()
    if choice == "credibility": return CredibilityWeightedMatcher()
    if choice == "roundrobin":  return RoundRobinMatcher()
    return NearestMatcher()
```

A.5 — Outbox-lite

Where: `sos-service/app/main.py trigger()` endpoint

The Outbox pattern solves a classic distributed-systems problem: we need to write to a database AND publish a Kafka event, but we can't wrap both in a single transaction because SQLite and Kafka are separate systems. The lite version keeps them in the same async block to make the window of inconsistency as small as possible (if the process crashes after the DB write but before the Kafka send, the event is lost). A full Outbox with a relay table would eliminate that window entirely.

```
# sos-service/app/main.py trigger()
iid = str(uuid.uuid4())
media_ref = body.media_ref or f"sim://incident/{iid}/blob.bin"

# Step 1: write to SQLite -- incident exists regardless of what happens next
insert_incident(iid, claims["sub"], body.lat, body.lon, body.mode, media_ref)

# Step 2: publish to Kafka -- if we crash here the incident is in the DB but
#         the event never fires. This is the outbox-lite risk.
await publish(
    "sos.triggered",
    {"incident_id": iid, "user_id": claims["sub"],
     "lat": body.lat, "lon": body.lon, "mode": body.mode},
    key=iid,
)
```

A.6 — Circuit Breaker [completed in this submission]

Where: `app/events.py` class `CircuitBreaker` and `publish()`

Without a circuit breaker, if the Kafka broker goes down every `publish()` call blocks until the TCP socket times out (up to 30 s). During that time the asyncio event loop can't handle any incoming HTTP requests, making the whole service unresponsive. The breaker detects repeated failures and starts returning immediately with an exception instead, keeping the HTTP API alive.

The three states are CLOSED (normal operation), OPEN (too many failures — fast fail immediately), and HALF_OPEN (cooling window has passed — try one probe to see if the broker is back).

```
# app/events.py CircuitBreaker

def allow_request(self) -> bool:
    if self._state == self.CLOSED:
        return True          # all good, go ahead
    if self._state == self.OPEN:
        self._try_reset()    # check if cooling window has passed
        # if _try_reset moved us to HALF_OPEN we return True for the probe
        return self._state == self.HALF_OPEN
    return True              # HALF_OPEN: single probe allowed

def on_success(self) -> None:
    # probe or normal call succeeded -> close the breaker, reset counter
    self._failures = 0
    self._state = self.CLOSED
    self._opened_at = None

def on_failure(self) -> None:
    self._failures += 1
    if self._state == self.HALF_OPEN:
        # probe failed -> reopen and restart the cooling clock
```

```

        self._state      = self.OPEN
        self._opened_at = time.monotonic()
    elif self._failures >= self._fail_max:
        # hit the threshold -> trip the breaker
        self._state      = self.OPEN
        self._opened_at = time.monotonic()

# every publish() call goes through the breaker:
async def publish(topic, payload, key=None):
    if not _breaker.allow_request():
        raise RuntimeError(f"circuit is OPEN – skipping '{topic}'")
    try:
        p = await producer()
        await p.send_and_wait(topic, value=payload, key=key)
        _breaker.on_success()
    except Exception:
        _breaker.on_failure()
        raise

```

Part B — Patterns Added

B.1 — CQRS (Command Query Responsibility Segregation)

Where: `analytics-service/` — the entire service is the read model

Catalogue: [Patterns of Enterprise Application Architecture \(Fowler 2002\)](#)

CQRS separates the code paths for writing data (commands) and reading it (queries). In HELEP the write side lives in `sos-service`, `dispatch-service`, and `notification-service`. The read side lives entirely in `analytics-service`.

The problem this solves: if analytics queries ran against the same SQLite databases as the write services, a slow GROUP BY over the incident log could hold a write lock and delay a real SOS trigger. By giving analytics its own database — kept up to date by consuming Kafka events — the two paths never touch each other.

```
# analytics-service/app/main.py

STREAMS = [
    "user.registered", "sos.triggered", "sos.cancelled",
    "responder.assigned", "responder.confirmed",
    "safety.zone.entered", "notification.sent",
]

async def on_event(p: dict) -> None:
    # write side of the read model: update analytics DB from events
    bump_event(p["_stream"])
    if p["_stream"] == "sos.triggered":
        log_incident(p["incident_id"], p["lat"], p["lon"], p["mode"])
    elif p["_stream"] == "safety.zone.entered":
        log_zone_hit(p["zone_id"], p["incident_id"])

# query endpoints -- never call any other service over HTTP
@app.get("/stats/events")
async def stats_events():
    return [dict(r) for r in event_summary()]

@app.get("/stats/zones")
async def stats_zones():
    return [dict(r) for r in zone_summary()]
```

Trade-off: analytics is eventually consistent. It reflects events a few milliseconds after they happen (Kafka delivery time), not instantaneously. That's fine for a heatmap dashboard. In exchange, the write path is completely unaffected by how many analysts are querying the dashboard at the same time.

B.2 — Bulkhead

Where: `app/events.py consume()` — the `group_id` argument in each service's `startup()`

Catalogue: [Release It! \(Nygard 2018\)](#) — resource isolation

The Bulkhead pattern isolates parts of a system so that a failure or slowdown in one section doesn't spill into another. In distributed systems it often means giving each consumer its own pool of connections or threads.

In HELEP it works through Kafka consumer groups. Each service passes its own `group_id` to `consume()`. Kafka tracks each group's offset independently. If the analytics consumer is processing a slow aggregation and falls 5000 messages behind, that has zero effect on the dispatch consumer's offset — it's reading from the same partition but keeping a completely separate position.

```
# dispatch-service/app/main.py startup()
asyncio.create_task(
    consume(
        ["sos.triggered", "sos.cancelled"],
        "dispatch-service",      # <-- independent group; will not be blocked
        on_event,                #      by any other service's lag
    )
)

# analytics-service/app/main.py startup()
asyncio.create_task(
    consume(
        STREAMS,
        "analytics-service",     # <-- its own offset tracking
        on_event,
    )
)
# If analytics falls 10000 messages behind, dispatch-service never notices.
```


Part C — Anti-pattern Avoided

C.1 — Shared Database (Distributed Monolith)

Anti-pattern class: Data Coupling / Distributed Monolith

Catalogue: *Microservices Patterns* (Richardson 2018) ch. 2

The shared database anti-pattern means multiple services read and write the same schema. The services look independent from the outside (separate processes, separate deployments) but they're tightly coupled at the data layer. A migration that renames a column in the dispatch table can break the analytics query that read it. A long analytics query can hold a lock that delays a write in sos-service. This is called a Distributed Monolith — you get the operational complexity of microservices with none of the isolation benefits.

HELEP avoids this deliberately. Every service has its own database file at a separate path, mounted on its own PVC, accessible only to that service's container. There are no shared connection strings, no cross-service JOINS, and no foreign keys across service boundaries. Cross-service communication uses Kafka events only.

```
# Each service opens its OWN database path -- nothing is shared:
# user-service:          DB_PATH = /data/user.db
# sos-service:           DB_PATH = /data/sos.db
# dispatch-service:      DB_PATH = /data/dispatch.db
# notification-service:  DB_PATH = /data/notification.db
# analytics-service:     DB_PATH = /data/analytics.db

# In the Helm sub-chart (templates/pvc.yaml), each service gets its own PVC:
# spec:
#   accessModes: ["ReadWriteOnce"] # one pod, one volume -- cannot be shared
#   storageClassName: standard
#   resources:
#     requests:
#       storage: 1Gi

# If analytics-service.db gets corrupted, sos-service is completely unaffected.
# You can run a migration on dispatch.db without touching anything else.
```

The Repository pattern (app/db.py) reinforces this: there is no import of another service's db module anywhere in the codebase. The only thing that flows between services is a Kafka event — a plain Python dict with string keys.